Dynamic Idempotent Smoke Test HOWTO Addendum

Purpose: make the smoke test adapt to the current platform state. The smoke runner must not be a fixed skeleton-only or organ-only script. It must discover safe test modules for whatever exists now: Kubernetes, Terraform, AgentField, RunPod, GRN, skeleton batches, organ batches, config integration, and future platform expansions.

1. Final model

Item	Meaning
`IDEMPOTENT_SMOKETEST_DYNAMIC.md`	Human/spec reference that defines the dynamic smoke protocol. Keep in `/workspace/docs/`. Do not prompt Codex with the whole file every time.
`/workspace/scripts/smoke_current_state.sh`	The single command entrypoint. Codex runs this after batches and before/after config integration.
`/workspace/tests/smoke.d/*.smoke.sh`	Discovered safe smoke modules. Each module tests one platform area only.
`/workspace/runs/smoke/<timestamp-phase>/SMOKE_REPORT.md`	Generated current-state smoke report. Read this before continuing.
`/workspace/runs/smoke/<timestamp-phase>/module-results/`	Per-module logs and outputs. Useful for debugging.

2. Directory layout

Path	Owner	Purpose
`/workspace/docs/IDEMPOTENT_SMOKETEST_DYNAMIC.md`	ChatGPT drafts; Codex writes	Protocol/spec for dynamic smoke testing.
`/workspace/scripts/smoke_current_state.sh`	Codex writes/updates	Orchestrator that discovers and runs modules.
`/workspace/tests/smoke.d/`	Codex writes/updates	Smoke module folder.
`/workspace/tests/smoke.d/10-workspace.smoke.sh`	Codex writes/updates	Verifies expected workspace and mount roots exist.
`/workspace/tests/smoke.d/20-python.smoke.sh`	Codex writes/updates	Verifies import/CLI/package basics if Python project exists.
`/workspace/tests/smoke.d/30-skeleton-contract.smoke.sh`	Codex writes/updates	Verifies skeleton output contract and safe dummy run if available.
`/workspace/tests/smoke.d/40-organ-contract.smoke.sh`	Codex writes/updates	Verifies organ dry-run contract only; no live organ action.
`/workspace/tests/smoke.d/50-kubernetes.smoke.sh`	Codex writes/updates when Kubernetes appears	Verifies manifests/config syntax only; no live cluster mutation by default.
`/workspace/tests/smoke.d/60-terraform.smoke.sh`	Codex writes/updates when Terraform appears	Runs safe formatting/validation only; no apply/destroy.
`/workspace/tests/smoke.d/70-agentfield.smoke.sh`	Codex writes/updates when AgentField surfaces exist	Verifies CLI/config/schema/routing basics.
`/workspace/tests/smoke.d/80-runpod.smoke.sh`	Codex writes/updates when RunPod surfaces exist	Verifies templates/env/config only; no pod launch unless explicitly allowed.
`/workspace/tests/smoke.d/90-grn.smoke.sh`	Codex writes/updates when GRN science exists	Verifies GRN safe dry-run or import/fixture tests.

Path	Owner	Purpose
`/workspace/tests/smoke.d/99-config-integration.smoke.sh`	Codex writes/updates after config integration exists	Verifies config/lv/workflow aliases and health-check definitions.

3. Smoke phases

Phase	When to run	What it should include
`skeleton-progress`	After every skeleton batch	Workspace, Python/package, skeleton contract, any existing platform modules.
`skeleton-complete`	After all skeleton batches	Same as progress, but stricter: required skeleton contract modules must PASS or WARN with accepted reason.
`organ-progress`	After every organ batch	Workspace, Python/package, skeleton contract, organ dry-run contract, GRN/organ modules if present.
`organ-complete`	After all organ batches	Same as organ progress, but stricter: organ dry-run and safety gates must be accounted for.
`pre-config`	Before vmuser/operator config integration	Validates evidence exists and no config integration is attempted from missing evidence.
`post-config`	After vmuser/operator config integration	Verifies config aliases, lv profiles, health checks, and workflow rows exist and are non-destructive.
`platform-current`	Anytime	Runs all discovered modules in current safe mode.

4. Module contract

Every module in `/workspace/tests/smoke.d/*.smoke.sh` must obey this contract.

Field	Exact requirement
Filename	`NN-domain.smoke.sh`, for example `60-terraform.smoke.sh`.
Executable	Must be runnable with `bash module.smoke.sh`.
Inputs	Reads `PHASE`, `BATCH_SLUG`, `PROJECT_ROOT`, `SMOKE_RUN_DIR`, and optional `ALLOW_LIVE=0/1`.
Writes	Only under `$SMOKE_RUN_DIR/module-results/` unless the module is explicitly a harmless local cache check.
Safe default	Must not mutate cloud, cluster, config internals, live organ state, secrets, or external services.
Missing dependency	Return SKIP or WARN, not FAIL, when the platform area is not present yet.
Real failure	Return FAIL only when the platform area is present and expected safe checks fail.
Idempotent	Re-running must not change project state except creating a new smoke report.
Output	Print short status lines and write module log into `$SMOKE_RUN_DIR/module-results/<module>.log`.

5. Status meaning

Status	Meaning	Continue?
PASS	The applicable check succeeded.	Yes.
SKIP	The platform area is not present/applicable yet.	Yes.
WARN	The platform area exists but optional evidence, optional tools, or non-blocking checks are missing.	Usually yes, but record the reason.
FAIL	Required current-state contract is broken.	No. Fix before continuing.

6. Minimal command usage

Situation	Command
After skeleton batch 01	`BATCH_SLUG="01-runtime-substrate" bash /workspace/scripts/smoke_current_state.sh skeleton-progress`
After any later skeleton batch	`BATCH_SLUG="<batch-slug>" bash /workspace/scripts/smoke_current_state.sh skeleton-progress`
After all skeleton batches	`bash /workspace/scripts/smoke_current_state.sh skeleton-complete`
After organ batch R01	`BATCH_SLUG="R01-<slug>" bash /workspace/scripts/smoke_current_state.sh organ-progress`
After all organ batches	`bash /workspace/scripts/smoke_current_state.sh organ-complete`
Before config integration	`bash /workspace/scripts/smoke_current_state.sh pre-config`
After config integration	`bash /workspace/scripts/smoke_current_state.sh post-config`
Anytime current-state check	`bash /workspace/scripts/smoke_current_state.sh platform-current`

7. Step ledger: D-SM1 - Create/update the dynamic smoke protocol

Field	Exact content
Step owner	ChatGPT drafts the protocol; Codex writes it into the project.
When	Once at the beginning, and later only when the smoke architecture changes.
ChatGPT prompt	Create or update `IDEMPOTENT_SMOKETEST_DYNAMIC.md` as the dynamic smoke-test protocol. It must define the orchestrator, module contract, safe/idempotent rules, statuses, phases, and domain modules for Kubernetes, Terraform, AgentField, RunPod, GRN, skeleton, organs, and config integration. Do not include project-specific secrets.
Upload to ChatGPT	Current `IDEMPOTENT_SMOKETEST.md` if present; current workflow PDFs or ledgers if needed; latest skeleton/organ workflow assumptions if changed.
ChatGPT creates	`IDEMPOTENT_SMOKETEST_DYNAMIC.md` content; optional addendum content.
Codex must have access to	`/workspace/docs/`; ChatGPT-created `IDEMPOTENT_SMOKETEST_DYNAMIC.md`.
Codex prompt	Write the provided `IDEMPOTENT_SMOKETEST_DYNAMIC.md` to `/workspace/docs/IDEMPOTENT_SMOKETEST_DYNAMIC.md`. Do not edit config. Do not run live actions.
Codex may run	`mkdir -p /workspace/docs`; `test -f /workspace/docs/IDEMPOTENT_SMOKETEST_DYNAMIC.md`; file write commands only.
Codex creates/updates	`/workspace/docs/IDEMPOTENT_SMOKETEST_DYNAMIC.md`.
Codex must not create	Config/lv/workflow edits; cloud resources; cluster resources; live organ runs.

8. Step ledger: D-SM2 - Create/update the smoke orchestrator

Field	Exact content
Step owner	Codex creates the script from the protocol; ChatGPT may draft the script if requested.
When	Once, then whenever module contract or reporting behavior changes.
ChatGPT prompt	Using `/workspace/docs/IDEMPOTENT_SMOKETEST_DYNAMIC.md`, draft `smoke_current_state.sh`. It must discover `/workspace/tests/smoke.d/*.smoke.sh`, create a timestamped report folder under `/workspace/runs/smoke/`, run modules in sorted order, pass `PHASE`, `BATCH_SLUG`, `PROJECT_ROOT`, `SMOKE_RUN_DIR`, and `ALLOW_LIVE=0`, collect PASS/WARN/SKIP/FAIL, and write `SMOKE_REPORT.md`.
Upload to ChatGPT	`/workspace/docs/IDEMPOTENT_SMOKETEST_DYNAMIC.md`; existing `/workspace/scripts/smoke_current_state.sh` if present; one existing `SMOKE_REPORT.md` if debugging report format.
ChatGPT creates	Optional `smoke_current_state.sh` draft; optional patch; optional expected report format.
Codex must have access to	`/workspace/docs/IDEMPOTENT_SMOKETEST_DYNAMIC.md`; `/workspace/scripts/`; `/workspace/tests/smoke.d/`; `/workspace/runs/smoke/`.

Field	Exact content
Codex prompt	Read `/workspace/docs/IDEMPOTENT_SMOKETEST_DYNAMIC.md`. Create or update `/workspace/scripts/smoke_current_state.sh` to match the protocol. Do not implement domain logic inside the orchestrator except discovery, environment setup, module execution, and report aggregation. Do not edit config.
Codex may run	`mkdir -p /workspace/scripts /workspace/tests/smoke.d /workspace/runs/smoke`; `chmod +x /workspace/scripts/smoke_current_state.sh`; `bash -n /workspace/scripts/smoke_current_state.sh`; `bash /workspace/scripts/smoke_current_state.sh platform-current`.
Codex creates/updates	`/workspace/scripts/smoke_current_state.sh`; `/workspace/runs/smoke/.../SMOKE_REPORT.md` when tested.
Codex must not create	Hardcoded skeleton-only runner; hardcoded organ-only runner; live cluster/cloud mutations; Terraform apply/destroy; config integration edits.

9. Step ledger: D-SM3 - Add/update smoke modules after each batch

Field	Exact content
Step owner	Codex updates modules; ChatGPT helps design module boundaries when needed.
When	After each skeleton batch, after each organ batch, and whenever a new platform surface appears.
ChatGPT prompt	Review the named files and tell me which `/workspace/tests/smoke.d/*.smoke.sh` modules should be added or updated. Do not repeat the full prompts from the files. Name the files Codex must read and the exact module files it should create or update.
Upload to ChatGPT	Latest batch `SPEC.md`; latest batch `RUN_INSTRUCTIONS.md`; latest `POSTCHECK.md`; latest `INTEGRATION_REQUEST.md`; latest `SMOKE_REPORT.md`; codebase analysis output if available; existing `/workspace/tests/smoke.d/` file list or module contents if changing modules.
ChatGPT creates	Module update plan; optional module content or patch; exact Codex file list; exact smoke command to run.
Codex must have access to	`/workspace`; latest batch files; `/mnt/egress/dev-recordings/skeleton/<batch-slug>/POSTCHECK.md` or `/mnt/egress/organs/dev-recordings/organs/<batch-slug>/POSTCHECK.md`; matching `INTEGRATION_REQUEST.md`; `/workspace/tests/smoke.d/`; `/workspace/scripts/smoke_current_state.sh`.
Codex prompt	Read the latest batch `SPEC.md`, `RUN_INSTRUCTIONS.md`, `POSTCHECK.md`, `INTEGRATION_REQUEST.md`, and existing `/workspace/tests/smoke.d/` modules. Add or update only the smoke modules needed for the new or changed platform surface. Keep modules safe and idempotent. Do not edit config. Then run the appropriate smoke command.
Codex may run	`bash -n /workspace/tests/smoke.d/*.smoke.sh`; `BATCH_SLUG="<batch-slug>" bash /workspace/scripts/smoke_current_state.sh skeleton-progress`; `BATCH_SLUG="<batch-slug>" bash /workspace/scripts/smoke_current_state.sh organ-progress`.
Codex creates/updates	One or more `/workspace/tests/smoke.d/NN-domain.smoke.sh` files; latest `/workspace/runs/smoke/.../SMOKE_REPORT.md`; optional module logs.
Codex must not create	Production resources; cluster changes; Terraform state changes; live RunPod launches; live organ actions; config/lv/workflow edits.

10. Step ledger: D-SM4 - Run the dynamic smoke test after each batch

Field	Exact content
Step owner	Codex runs; user/ChatGPT reviews result.
When	Immediately after each skeleton/organ batch has written POSTCHECK.md and INTEGRATION_REQUEST.md.
ChatGPT prompt	Read the latest `SMOKE_REPORT.md`, `POSTCHECK.md`, and `INTEGRATION_REQUEST.md`. Decide whether the current state is PASS, WARN acceptable, WARN blocking, or FAIL. Name exact missing files if any. Do not invent missing evidence.
Upload to ChatGPT	Latest `/workspace/runs/smoke/.../SMOKE_REPORT.md`; latest batch `POSTCHECK.md`; latest batch `INTEGRATION_REQUEST.md`; module logs only if FAIL/WARN is unclear.
ChatGPT creates	PASS/WARN/FAIL decision; fix request if needed; companion-update recommendation if state is checked.
Codex must have access to	`/workspace/scripts/smoke_current_state.sh`; `/workspace/tests/smoke.d/`; `/workspace`; current evidence roots.
Codex prompt	Run the dynamic smoke test for the current batch using the correct phase and `BATCH_SLUG`. Do not change source code except if explicitly asked to fix a smoke failure. Do not edit config. Return the path to `SMOKE_REPORT.md`.
Codex may run	Skeleton: `BATCH_SLUG="<batch-slug>" bash /workspace/scripts/smoke_current_state.sh skeleton-progress`; Organ: `BATCH_SLUG="<batch-slug>" bash /workspace/scripts/smoke_current_state.sh organ-progress`.
Codex creates/updates	`/workspace/runs/smoke/<timestamp-phase>/SMOKE_REPORT.md`; `/workspace/runs/smoke/<timestamp-phase>/module-results/*.log`.

Field	Exact content
Codex must not create	New smoke modules during a run-only request unless explicitly asked; config edits; live external resources.

11. Step ledger: D-SM5 - Add domain-specific module safely

Field	Exact content
Step owner	ChatGPT can draft; Codex writes and runs.
When	When a new domain appears, such as Kubernetes, Terraform, AgentField, RunPod, GRN, config integration, or future infra/science module.
ChatGPT prompt	Using the uploaded domain files, draft a safe idempotent smoke module named `/workspace/tests/smoke.d/NN-domain.smoke.sh`. It must SKIP when the domain is absent, PASS when safe checks succeed, WARN for optional missing tools/evidence, and FAIL only for broken required current-state contracts. Do not include apply, deploy, launch, mutate, delete, or live calls.
Upload to ChatGPT	Domain files only: for Kubernetes, manifests/helm/kustomize file list; for Terraform, `.tf` files and module layout; for AgentField, config/schema/CLI docs; for RunPod, template/config/env docs with secrets removed; for GRN, safe fixture/CLI docs; existing smoke modules if style must match.
ChatGPT creates	One module draft; exact safe commands; exact forbidden commands; expected PASS/WARN/SKIP/FAIL behavior.
Codex must have access to	Domain source files in `/workspace`; `/workspace/tests/smoke.d/`; `/workspace/scripts/smoke_current_state.sh`.
Codex prompt	Write the provided module to `/workspace/tests/smoke.d/NN-domain.smoke.sh`. Run `bash -n` on it. Run the dynamic smoke test in `platform-current` or current batch phase. Do not run forbidden live/mutating commands.
Codex may run	`bash -n /workspace/tests/smoke.d/NN-domain.smoke.sh`; `bash /workspace/scripts/smoke_current_state.sh platform-current`; safe static commands only, such as format/check/validate/dry-run with no remote mutation.
Codex creates/updates	`/workspace/tests/smoke.d/NN-domain.smoke.sh`; smoke report and module logs.
Codex must not create	Secrets; credentials; cloud resources; cluster resources; Terraform state changes; live RunPod pods; real organ outputs unless phase explicitly allows guarded real action.

12. Recommended safe checks by domain

Domain	Safe checks	Forbidden by default
Workspace	`test -d /workspace`; check expected mount roots; check write only in `/workspace/runs/smoke`.	Deleting workspace contents.
Python/package	Import package; `python -m compileall`; safe CLI help; fixture dry-run.	Installing global packages without request; network calls.
Skeleton	Dummy CLI run; output file existence; schema/filename contract check.	Config edits; real organ actions.
Organs	Dry-run only; safety gate check; output contract check.	Live organ action without explicit guard and request.
Kubernetes	YAML parse; `kubectl --dry-run=client` if available; `helm template` if available.	`kubectl apply/delete`; changing cluster context.
Terraform	`terraform fmt -check`; `terraform validate` after local init only if safe; no backend mutation.	`terraform apply`; `terraform destroy`; state migration.
AgentField	CLI help; schema validation; config parse; route table dry check.	Editing operator config unless in config-integration phase.
RunPod	Template/env shape validation; secret presence check without printing values.	Launching pods; printing secrets; modifying account resources.
GRN	Safe fixture run; import check; deterministic tiny dry-run.	Expensive run; external data pull; live training/production action.
Config integration	Alias/profile/health-check presence; no-op status command.	Integrating config outside dedicated vmuser/operator config batch.

13. Rule for continuing

Result	Next action
All PASS/SKIP	Continue to next planned batch or companion update.
WARN only	Decide whether WARN is acceptable; document reason in POSTCHECK.md or companion.
FAIL	Stop. Fix the failing module or platform issue. Re-run smoke before continuing.
Missing required evidence	Stop. Ask for the exact missing file. Do not guess.